Problem Statement: AI-Powered Smart To-Do Application

Objective

Build a **FastAPI-based Smart To-Do Application** where users can enter tasks in simple, incomplete, or broken sentences. The application should use an AI model in the backend to understand the user's intention, rewrite the task in a clear and detailed format, and automatically create a structured to-do item.

Problem

Users do not always create tasks in a structured manner.

For example, a user might enter:

"finish report by friday and send to manager"

or:

"call client tomorrow morning regarding project"

or:

"need to check sales data and prepare something for meeting"

Instead of asking the user to manually provide a title, description, priority, deadline, and other details, the application should use an **LLM through an API** to understand the input and generate a structured task.

Application Requirements

The application should provide a FastAPI backend with an endpoint such as:

`POST /tasks`

The user sends a simple message:

```
{
  "message": "finish the sales report by friday and send it to the manager"
}
```

The backend should:

1. Receive the user's input through FastAPI.
2. Send the input to an AI model.
3. Use an appropriate **system prompt** to instruct the AI to act as a task-management assistant.
4. Ask the AI to convert the user's message into a structured task.
5. Receive the AI-generated response.
6. Store the generated task.
7. Return the structured task to the user.

Expected AI Output

The AI should transform the input into something similar to:

```
{
  "title": "Complete Sales Report",
  "description": "Prepare the sales report, review the required sales data, finalize the report, and send the completed report to the manager.",
  "priority": "High",
  "due_date": "Friday",
  "status": "Pending"
}
```

The exact fields can be decided by the students.

Suggested API Endpoints

Create Task

`POST /tasks`

Accepts a natural-language task and uses AI to generate a structured task.

Get Tasks

`GET /tasks`

Returns all previously created tasks.

Get Task

`GET /tasks/{task_id}`

Returns a specific task.

Delete Task

`DELETE /tasks/{task_id}`

Deletes a task.

AI Integration

The AI should be responsible for **understanding and structuring the user's natural-language input**.

Students should design a suitable system prompt such as:

You are an AI task management assistant.

Convert the user's informal task description into a structured task.

Extract:
- Task title
- Detailed description
- Priority

- Due date if mentioned
- Status

Return the result in JSON format.
Do not invent information that is not present in the user's input.

The application should then pass the user's message as the **user message**.

Database

For simplicity, students can use **SQLite** to store the tasks.

Each task can contain:

- `id`
- `title`
- `description`
- `priority`
- `due_date`
- `status`
- `created_at`

Example User Flow

User Input:

"tomorrow call John about the payment issue and ask him to send the invoice"

FastAPI → AI

The backend sends the instruction and user message to the LLM.

AI → FastAPI

The AI returns structured information:

```
{
  "title": "Call John About Payment Issue",
  "description": "Contact John regarding the payment issue and request that he send the invoice.",
  "priority": "Medium",
  "due_date": "Tomorrow",
  "status": "Pending"
}
```


FastAPI → Database

The structured task is stored in SQLite.

FastAPI → User

The API returns the newly created task.

Minimum Deliverables

Students should build:

- FastAPI application
- At least 4 REST API endpoints
- AI integration using an LLM API
- System message + user message implementation
- Structured AI output
- SQLite database
- Pydantic models for request/response validation
- Swagger/OpenAPI documentation
- Proper error handling

Bonus Challenges

Students can optionally extend the application with:

- AI-generated task priority
- Natural-language date extraction

- Update tasks using natural language
- Search tasks using natural-language queries
- Mark tasks as completed using AI
- Simple HTML/JavaScript frontend
- Authentication
- Logging every user request and AI response
- Store the AI-generated reasoning metadata without exposing private chain-of-thought
- Add an endpoint such as `POST /tasks/ai-command` where users can say:

"Complete my sales report task"

and the backend identifies and updates the appropriate task.

Final Goal

The goal is **not simply to build a To-Do API**.

The goal is to demonstrate how a traditional **FastAPI backend can be enhanced with Generative AI**, allowing users to interact with an application using natural language while the backend converts that unstructured input into structured, actionable data.